

算法基础 Homework 5

郝思粤

学号 PB23151779

中国科学技术大学

2025 年 11 月 30 日

Q1. ($2 \times 10 = 20$ 分)**(a) 求矩阵链最优括号化方案 (给出计算过程)**

给定维度序列

$$p = (5, 10, 3, 12, 5, 50, 6),$$

对应 6 个矩阵:

$$A_1 : 5 \times 10, A_2 : 10 \times 3, A_3 : 3 \times 12, A_4 : 12 \times 5, A_5 : 5 \times 50, A_6 : 50 \times 6.$$

令

$$m[i, j] = \text{计算 } A_i A_{i+1} \cdots A_j \text{ 的最小标量乘法次数}, \quad m[i, i] = 0.$$

递推式

$$m[i, j] = \min_{i \leq k < j} (m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j).$$

下面按链长 L 计算。

链长 $L = 2$

$$m[1, 2] = 5 \cdot 10 \cdot 3 = 150,$$

$$m[2, 3] = 10 \cdot 3 \cdot 12 = 360,$$

$$m[3, 4] = 3 \cdot 12 \cdot 5 = 180,$$

$$m[4, 5] = 12 \cdot 5 \cdot 50 = 3000,$$

$$m[5, 6] = 5 \cdot 50 \cdot 6 = 1500.$$

链长 $L = 3$

$$m[1, 3] = \min\{0 + 360 + 5 \cdot 10 \cdot 12, 150 + 0 + 5 \cdot 3 \cdot 12\} = 330,$$

$$m[2, 4] = 330, \quad m[3, 5] = 930, \quad m[4, 6] = 1860.$$

链长 $L = 4$

$$m[1, 4] = 405, \quad m[2, 5] = 2430, \quad m[3, 6] = 1770.$$

链长 $L = 5$

$$m[1, 5] = 1655, \quad m[2, 6] = 1950.$$

链长 $L = 6$ (完整矩阵链)

$$\begin{aligned} m[1, 6] &= \min\{0 + 1950 + 5 \cdot 10 \cdot 6, 150 + 1770 + 5 \cdot 3 \cdot 6, 330 + 1860 + 5 \cdot 12 \cdot 6, \\ &\quad 405 + 1500 + 5 \cdot 5 \cdot 6, 1655 + 0 + 5 \cdot 50 \cdot 6\} \\ &= \boxed{2010}. \end{aligned}$$

故最优值 $m[1, 6] = 2010$ 。

回溯割点，可得到括号结构：

$$(A_1 A_2)((A_3 A_4)(A_5 A_6)),$$

总乘法次数 2010。

(b) 证明：对 n 个元素的表达式完全括号化，恰好需要 $n - 1$ 对括号

用归纳法。

基例： $n = 1$ 时只有一个元素，不需要括号， $0 = n - 1$ 成立。

归纳假设：对任意 $k < n$ ，含 k 个元素的完全括号化需要 $k - 1$ 对括号。

归纳步骤：考虑含 n 个元素的完全括号化，最外层一定是

$$(E_L E_R),$$

设左、右表达式分别含 k 和 $n - k$ 个元素。根据假设, E_L 需要 $k - 1$ 对括号, E_R 需要 $(n - k) - 1$ 对括号, 外层再加一对, 总数

$$(k - 1) + (n - k - 1) + 1 = n - 1.$$

因此对 n 也成立, 命题得证。

Q2 (20 分)

参考提示为, 设

$f(k, m)$ = 在最多 m 次实验、 k 个蛋的情况下, 最多可确定的楼层数。

一次实验有两种结果:

- 蛋碎: 变成 $k - 1$ 个蛋、 $m - 1$ 次实验, 能确定 $f(k - 1, m - 1)$ 层;
- 不碎: 还是 k 个蛋、 $m - 1$ 次实验, 能确定 $f(k, m - 1)$ 层。

再加上当前这一层, 有

$$f(k, m) = f(k - 1, m - 1) + f(k, m - 1) + 1.$$

边界:

$$f(0, m) = 0, \quad f(k, 0) = 0.$$

求最少实验次数: 从 $m = 0, 1, 2, \dots$ 依次往上, 对每个 m 用上式滚动计算 $f(1, m), \dots, f(k, m)$, 直到 $f(k, m) \geq n$ 为止, 此时的 m 即答案。每个 m 只算 k 个状态, 所以总复杂度 $O(km^*)$, 其中 m^* 为最终答案。

进一步可以证明

$$f(k, m) = \sum_{i=1}^k \binom{m}{i},$$

它是关于 m 的 k 次多项式, 所以大致有 $m^* = \Theta(n^{1/k})$, 从而时间 $O(km^*) = O(n^{1/k})$, 确实小于 $O(n)$ 。

Q3 (20 分)

给定长度为 n 的数组 $nums[1..n]$, 每个数在 $[1, n]$, 要求找一个最长子序列, 使得:

- 严格递增;
- 相邻元素差不超过 k 。

状态:

$dp[i]$ = 以第 i 个元素为结尾的、满足条件的最长子序列长度.

初值 $dp[i] = 1$ 。

转移: 对每个 i , 枚举 $j < i$, 若

$$nums[j] < nums[i], \quad nums[i] - nums[j] \leq k,$$

则可以接在后面:

$$dp[i] = \max(dp[i], dp[j] + 1).$$

最后答案为 $\max_i dp[i]$ 。双重循环, 时间复杂度 $O(n^2)$, 空间 $O(n)$ 。

Q4 (20 分) 叠叠乐

(a)

把每本书看成一个二元组 (a_i, b_i) , a_i 是长边, b_i 是短边, 且 $a_i > b_i$ 。若书 i 放在书 j 上面, 需要

$$a_i < a_j, \quad b_i < b_j.$$

为方便处理, 先把所有书放到数组 $A[1..n]$ 中, 然后按

a 降序, a 相同时 b 降序

排序。排序后有

$$A[1].a \geq A[2].a \geq \cdots \geq A[n].a,$$

且同一长边里短边也是非增的。

接着做 DP。设

$C[i]$ = 以第 i 本书为最上层时，最多能叠的层数，

初值 $C[i] = 1$ 。对每个 i ，枚举 $j < i$ ，若

$$A[i].a < A[j].a, \quad A[i].b < A[j].b,$$

则可以把 i 放到 j 上面：

$$C[i] = \max(C[i], C[j] + 1).$$

最后结果为 $\max_i C[i]$ 。排序 $O(n \log n)$ ，DP 双循环 $O(n^2)$ ，总复杂度为 $O(n^2)$ 。

(b)

第 i 种积木尺寸为 $a_i \times b_i \times c_i$ ，可以任意旋转。对每一种尺寸，我取出三块，固定成三种摆放方式：

$$a_i \times b_i \text{ (高 } c_i), \quad a_i \times c_i \text{ (高 } b_i), \quad b_i \times c_i \text{ (高 } a_i).$$

对每种摆放方式，把底面两条边从大到小排序成 (ℓ, w) ， $\ell \geq w$ ，这样一共得到至多 $3n$ 个“虚拟积木”，每一个都有一个底面 (ℓ, w) 。

上层积木的底面必须在两个方向都严格小于下层底面，即

$$(\ell_2, w_2) \text{ 可以放在 } (\ell_1, w_1) \text{ 上} \iff \ell_2 < \ell_1, w_2 < w_1.$$

这就和 (a) 完全一样，只是物品个数变成 $3n$ 。因此对这 $3n$ 个二元组 (ℓ, w) 套用 (a) 中的排序 + DP，得到的最长长度就是最多能叠的积木数。时间复杂度仍是 $O((3n)^2) = O(n^2)$ 。

Q5 (2 × 10 = 20 分)

(a) 一维最大连续子数组

设

$f(i)$ = 所有从位置 i 开始的连续子数组中，和的最大值， $1 \leq i \leq n$.

考虑第 i 个位置时，要么单独从这里重新开始，要么把后面的最优段接上：

$$f(i) = \begin{cases} A[i], & f(i+1) < 0, \\ A[i] + f(i+1), & f(i+1) \geq 0. \end{cases}$$

边界 $f(n) = A[n]$ 。从 $i = n - 1$ 一直算到 1，即可得到所有 $f(i)$ ，答案是

$$\max_{1 \leq i \leq n} f(i).$$

如果想恢复具体区间，可以记录取得最大值的起点 i^* ，然后从 i^* 开始往右累加，直到再加会变小为止。整体时间 $O(n)$ 。

(b) 二维最大子矩阵

矩阵为 $M[1..n][1..n]$ 。套路是“先压成一维，再用上一问的算法”。

第一步：行前缀和。对每一行做前缀和

$$S[i][j] = \sum_{t=1}^j M[i][t], \quad 1 \leq i, j \leq n,$$

这样任意一行在列区间 $[c, d]$ 上的和可以 $O(1)$ 算出：

$$\sum_{t=c}^d M[i][t] = S[i][d] - S[i][c-1].$$

第二步：枚举左右列，把二维压成一维。用两重循环枚举所有列区间 $[c, d]$ 。对固定一对 (c, d) ，对每一行 k 定义

$$C[k] = S[k][d] - S[k][c-1], \quad 1 \leq k \leq n.$$

这相当于把列区间 $[c, d]$ 里每一行的和压成了一个一维数组 $C[1..n]$ 。

第三步：在 C 上跑 (a)。 在数组 C 上用 (a) 中的 $f(i)$ 算法求最大连续子数组和（顺便可以记录对应的起止行）。这个子数组对应的就是当前列区间 $[c, d]$ 下的最佳子矩阵。对所有 (c, d) 取最大即可。

复杂度：前缀和 $O(n^2)$ ；列区间有 $O(n^2)$ 种，每种需要 $O(n)$ 时间构造 C 并做一次一维 DP，所以总时间是

$$O(n^2) \cdot O(n) = O(n^3).$$